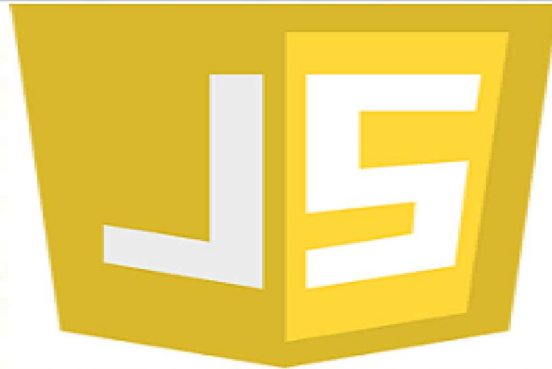


JAVASCRIPT



STRUCTURES DE CONTRÔLE

Structures de contrôle

- On utilise **if** pour exécuter du code si une condition est vérifiée.
- On peut ajouter un « sinon », appelé **else**, pour exécuter du code différent si la condition n'est pas vérifiée.

```
let age = parseInt(prompt("Donner votre age"));

if (age >= 18) {
  console.log ("Vous pouvez conduire !");
}
else {
  console.log ("Désolé, vous pourrez seulement conduire dans "
+ (18 - age) + "ans.");
}
```

Structures de contrôle

- Pour des conditions multiples, possible d'utiliser else if :

```
let age = 20;
if (age >= 24) {
  console.log("Vous pouvez voter et devenir sénateur !");
}
else if (age >= 18) {
  console.log("Vous pouvez voter.");
}
else if (age >= 16) {
  console.log("Vous ne pouvez pas voter mais vous pouvez"
+ "travailler sous certaines conditions.");
}
else {
  console.log("Vous devez être à l'école ?");
}
```

Pr. Ghazouani Mohamed

Programmation Web II 2024-2025

32

32

Structures de contrôle

- Possible de supprimer les accolades lorsqu'on n'a qu'une seule instruction :

```
let age = 20;
if (age >= 24)
  console.log("Vous pouvez voter et devenir sénateur !");
else if (age >= 18) {
  console.log("Vous pouvez voter.");
  console.log("Félicitations !");
}
else if (age >= 16)
  console.log("Vous ne pouvez pas voter mais vous pouvez"
+ "travailler sous certaines conditions.");
else
  console.log("Vous pouvez... aller à l'école ?");
```

Pr. Ghazouani Mohamed

Programmation Web II 2024-2025

33



33

Structures de contrôle

Alternative

L'opérateur ternaire est une façon concise d'écrire une condition if-else.
Il utilise le symbole ? et a la structure suivante :

```
variable = condition ? valeur_si_vrai : valeur_si_faux;
```

Voici un exemple simple :

```
let civilite = (genre == "F") ? "Madame" : "Monsieur";
```

Elle est équivalente a :

```
if (genre == "F")  
    civilite = "Madame";  
else  
    civilite = "Monsieur";
```

Pr. Ghazouani Mohamed

Programmation Web II 2024-2025



34

34

Structures de contrôle

Opérateurs logiques sur des booléens :

- **Opérateur ET : a && b**

true si **a** et **b** tous les deux vrais, **false** sinon. Si **a** est **false**, JavaScript n'essaie même pas d'évaluer **b**, car il sait que l'expression ne pourra être vraie de toute façon : renvoie directement **false**.

- **Opérateur OU : a || b**

true si soit **a** soit **b** vrai, **false** sinon. Si **a** est **true**, JavaScript n'essaie même pas d'évaluer **b**, car il sait qu'il sait que l'expression ne pourra être fausse de toute façon : renvoie directement **true**.

- **Opérateur NON : !a**

Renvoie **true** si **a** vaut **false** et inversement.

Pr. Ghazouani Mohamed

Programmation Web II 2024-2025

35

35

Structures de contrôle

JavaScript propose des opérateurs qui vous permettent de comparer des valeurs.

Opérateur	Description
==	Pour tester une égalité entre deux valeurs.
===	Pour tester une égalité entre deux valeurs en prenant aussi en considération le type des valeurs (chaîne, nombre, date).
!= ou <>	Pour tester la différence entre deux valeurs.
!==	Pour tester la différence entre des valeurs ou des types différents.
<	Pour tester si une valeur est strictement inférieure à une autre.
<=	Pour tester si une valeur est inférieure ou égale à une autre.
>	Pour tester si une valeur est strictement supérieure à une autre.
>=	Pour tester si une valeur est supérieure ou égale à une autre.
+=	Pour concaténer. L'expression a+=6 est équivalent à a=a+6.

Pr. Ghazouani Mohamed

Programmation Web II 2024-2025

36

36

Structures de contrôle

Exercice 1 (Utiliser un IF ternaire):

Si une personne est âgée de 18 ans et plus c'est une personne majeure, sinon c'est une personne mineure.

Exercice 2 :

Écrivez un programme en JavaScript qui demande à l'utilisateur de saisir une note (grade) comprise entre 0 et 100. En fonction de la note saisie, le programme doit afficher une alerte avec la mention correspondante :

- A** si la note est supérieure ou égale à 90.
- B** si la note est comprise entre 80 et 89.
- C** si la note est comprise entre 70 et 79.
- D** si la note est comprise entre 60 et 69.
- F** si la note est inférieure à 60.

Pr. Ghazouani Mohamed

Programmation Web II 2024-2025

37

37

Structures de contrôle

Choix multiple

L'instruction switch :

```
switch(x) {  
  case 1 :  
    instructions 1;  
    break;  
  case 2 :  
    instructions 2;  
    break;  
  ...  
  case n :  
    instructions 3;  
    break;  
  default : instructionsDefault;  
};
```

Pr. Ghazouani Mohamed

Programmation Web II 2024-2025

38

38

Structures de contrôle

Choix multiple

```
<script>  
let nb1, nb2  
let op  
nb1 = parseInt(prompt("donner nombre 1"))  
nb2 = parseInt(prompt("donner nombre 2"))  
op = prompt("donner operateur (+, -, *, /)")  
  
switch(op) {  
  case "+":  
    document.write(nb1+nb2);  
    break;  
  case "-":  
    document.write(nb1-nb2);  
    break;  
  case "*":  
    document.write(nb1*nb2);  
    break;  
  case "/":  
    {  
      if (nb2 == 0)  
        document.write("Opération indéterminée")  
      else  
        document.write(nb1/nb2)  
    }  
    break;  
  default:  
    document.write("Opération inexistante");  
}  
</script>
```

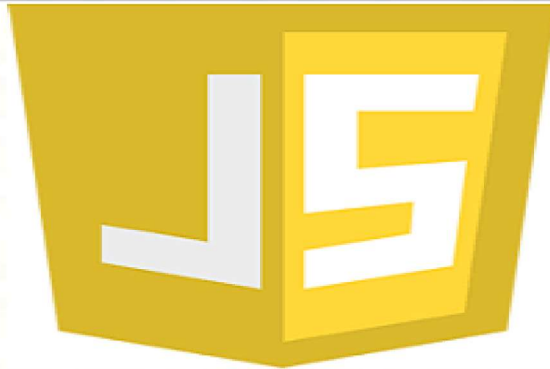
Pr. Ghazouani Mohamed



39

39

JAVASCRIPT



LES BOUCLES

Pr. Ghazouani Mohamed

Programmation Web II 2024-2025

40

40

Les boucles

- Boucles utilisées pour répéter du code plusieurs fois
- Utile pour ne pas réécrire la même chose de nombreuses fois, et souvent nécessaire car le nombre de fois qu'on veut répéter le code peut dépendre d'un paramètre dont la valeur ne peut être prédite
- Boucle **while** (« tant que ») : répète le code tant que la condition entre parenthèses est vraie

```
let bananes = 10;
while (bananes >= 1) {
  console.log ("J'ai " + bananes + " bananes et j'en mange une...");
  bananes--;
}
```

```
console.log ("Je n'ai plus de bananes.");
```

- Au passage, `bananes--` décrémente (= soustrait 1 à) la variable `bananes`, équivalent à `bananes = bananes - 1` ou `bananes -= 1`. De la même manière, `bananes++` incrémente (ajoute 1 à) la variable `bananes`.

Pr. Ghazouani Mohamed

Programmation Web II 2024-2025

41

41

Les boucles

La boucle do...while

- La boucle do...while en JavaScript exécute le bloc de code au moins une fois, puis continue de l'exécuter tant que la condition spécifiée est vraie.
- Ce type de boucle peut être utile pour des scénarios où tu veux t'assurer qu'une certaine condition est remplie avant de continuer avec le reste de ton programme.

```
do
{
    // Le code à exécuter
}
while (condition);
```

Les boucles

À votre avis, quel est le problème de ce script ?

```
let bananes = 10;
while (bananes >= 1) {
    console.log ("J'ai " + bananes + " bananes et j'en mange une...")
}
```

- **Boucle infinie !** Il faut toujours au moins que quelque chose « change » dans votre boucle pour qu'elle ait une chance de se terminer...
- C'est nécessaire mais pas suffisant ! Par exemple, si vous incrémentez la variable bananes au lieu de la décrémenter, vous « changerez » quelque chose, mais quand même une boucle infinie...
- Page qui ne répond pas, peut-être un message d'erreur au bout d'un moment...

Les boucles

- **Boucle for (« pour ») :** généralement, répète le code pour un nombre de fois donné, e.g.

```
for (let i = 0 ; i < 100 ; i++)  
    console.log (i);  
    console.log ("J'ai compté de 0 à 99 !");
```

- Une boucle for a 3 expressions séparées par des points-virgule :
for (à exécuter au départ ;
test : on reste dans la boucle ? (effectué aussi avant d'entrer) ;
à exécuter à la fin de chaque itération)

- On peut mettre plusieurs instructions séparées par des virgules pour chaque expressions... Du coup on peut faire des choses plus compliquées :

```
for (let i = 0, j = 0 ; i + j < 100 ; i++, j += 2)  
    document.write(i + " " + j + "<br />");
```

Pr. Ghazouani Mohamed

Programmation Web II 2024-2025

*Au passage, remarquez que pour les **for** aussi on peut supprimer les accolades si on n'a qu'une seule instruction !*
Marche pour les if, else if, else, while, for... à peu près pour n'importe quel bloc sauf les **fonctions** !

```
▪ break sort d'une boucle prématurément :  
for (let i = debut; i <= fin; i++) {  
    console.log("Je teste " + i); // Arrêt au premier multiple de 7  
    if (i % 7 == 0) {  
        console.log("Trouvé ! " + i);  
        break; // Sort de la boucle  
    }  
}
```

44

Les boucles

Exercice "Fizz Buzz"

L'exercice du "Fizz Buzz" est un problème couramment utilisé dans les entrevues techniques pour évaluer les compétences de programmation de base des candidats.

- Commencez à partir de 1 et comptez jusqu'à 50.
- Pour chaque nombre dans la séquence, effectuez les vérifications suivantes :
 - Si le nombre est divisible par 3, affichez "Fizz" à la place du nombre.
 - Si le nombre est divisible par 5, affichez "Buzz" à la place du nombre.
 - Si le nombre est à la fois divisible par 3 et par 5, affichez "Fizz Buzz" à la place du nombre.
 - Sinon, affichez simplement le nombre lui-même.

Voici un exemple des premiers éléments de la séquence "Fizz Buzz" jusqu'à 15 :

```
1  
2  
Fizz  
4  
Buzz  
Fizz  
7  
8  
Fizz  
Buzz  
11  
Fizz  
13  
14  
Fizz Buzz
```

Pr. Ghazouani Mohamed

Programmation Web II 2024-2025

45

Les boucles

Exercice "Fizz Buzz" - Solution

```
for (let i = 1; i <= 50; i++)  
{  
    if (i % 3 == 0 && i % 5 == 0)  
        console.log ("FizzBuzz");  
    else if (i % 3 == 0)  
        console.log ("Fizz");  
    else if (i % 5 == 0)  
        console.log ("Buzz");  
    else  
        console.log (i);  
}
```

Pr. Ghazouani Mohamed

Programmation Web II 2024-2025



46

46

Les boucles

Exercice : Trouver la somme des nombres

Écrivez un programme JavaScript qui utilise une boucle **while** pour calculer la somme des nombres de 1 à un nombre donné.

Étapes à suivre :

- Demandez à l'utilisateur d'entrer un nombre positif (par exemple : 5).
- Utilisez une boucle while pour calculer la somme des nombres de 1 jusqu'à ce nombre inclus.
- Affichez le résultat final dans la console.

Pr. Ghazouani Mohamed

Programmation Web II 2024-2025



47

47